SteemTwist

Steem with a Twist

White Paper — Version 0.1

https://puncakbukit.github.io/steemtwist

Published under the MIT License

1. Executive Summary

SteemTwist is a fully decentralised, serverless microblogging decentralised application (dApp) built on the Steem blockchain. It combines the familiar short-form social experience of platforms like Twitter/X with the permanent, censorship-resistant guarantees of a public blockchain. Every post, reply, vote, and encrypted message is a native Steem operation — stored immutably on-chain without any proprietary back-end, database, or cloud infrastructure.

The application ships as four static files (index.html, app.js, components.js, blockchain.js) hosted on GitHub Pages and is served entirely from the client's browser. User authentication and transaction signing are handled exclusively by Steem Keychain, so the application never touches a private key.

Key design principles: zero-backend architecture, censorship-resistance, end-to-end encrypted private messaging, an open JavaScript sandbox for programmable interactive posts (***Live Twists***), and zero-payout economics that separate social reputation from financial speculation.

2. Introduction

2.1 Problem Statement

Centralised microblogging platforms retain full authority over content moderation, data ownership, and feature availability. A single policy change or account suspension can erase years of a user's posts. Server outages, regulatory pressure, or acquisition can terminate an entire platform

overnight. Users have no cryptographic proof of authorship and no mechanism to verify that historical records have not been altered.

2.2 The SteemTwist Approach

SteemTwist stores all content as Steem comment operations. Because every Steem block is replicated across a distributed network of witness nodes and is permanently referenced by its cryptographic hash, posts written today remain readable and verifiable in perpetuity — independent of any company, server, or domain name.

The client-side-only architecture means SteemTwist can be mirrored, forked, or served from any static host (IPFS, Arweave, a personal web server) without loss of data access. The Steem blockchain is the only required back-end.

2.3 Community Identity

A SteemTwist user is called a **Twister** □. Short-form posts are called **Twists**; replies to Twists are **Thread Replies**; upvotes are **Twist Love**; resteems are **Retwists**; and interactive JavaScript posts are **Live Twists** ⚡ .

3. Architecture

3.1 Zero-Backend Design

SteemTwist has no proprietary server, API gateway, or database. The entire application is four static files:

File	Responsibility
index.html	HTML shell, CDN script tags with SRI hashes, CSS design tokens, Vue app mount point
blockchain.js	All Steem API and Keychain helpers — no Vue dependency
components.js	Vue 3 single-file components compiled at runtime from CDN
app.js	Vue Router views, root App component, global state management

3.2 Technology Stack

Layer	Technology
Blockchain	steem-js (Steem RPC client)
Signing & encryption	Steem Keychain browser extension
UI framework	Vue 3 CDN + Vue Router 4 CDN
Markdown rendering	marked.js CDN

HTML sanitisation	DOMPurify CDN
Hosting	GitHub Pages (static, no server configuration required)

All CDN scripts are loaded with integrity (SRI) and crossorigin="anonymous" attributes. A tampered CDN file is rejected by the browser before execution, providing supply-chain protection without a build pipeline.

3.3 Data Model

SteemTwist imposes a thin convention on top of standard Steem comment operations. Each calendar month the @steemtwist account publishes two root posts:

Root permlink	Purpose
feed-YYYY-MM	Monthly Twist feed — all twists are direct replies to this post
secret-YYYY-MM	Monthly Secret Twist feed — encrypted messages are posted here

Twists are Steem comment operations posted as children of the monthly feed root. Thread Replies are children of individual Twists. This two-level hierarchy makes the entire feed retrievable with a single getContentReplies RPC call.

Permlink naming conventions:

- Twist / flag reply: tw-YYYYMMDD-HHMMSS-username
- Secret Twist: st-YYYYMMDD-HHMMSS-username
- Monthly feed root: feed-YYYY-MM
- Monthly secret root: secret-YYYY-MM

3.4 RPC Fallback Chain

All blockchain calls use an automatic node-rotation strategy. If a request fails the client advances to the next node for the remainder of the session:

- https://api.steemit.com (primary)
- https://api.justyy.com
- https://steemd.steemworld.org
- https://api.steem.fans

3.5 Routing

Vue Router 4 uses hash history (createWebHashHistory), making all routes work without server-side URL rewriting — essential for static hosting on GitHub Pages.

Route	View	Description

/	HomeView	Personalised feed — followed Twisters; Firehose; Understream toggle
/explore	ExploreView	Global Twist Stream; sort New/Hot/Top; Firehose; Understream toggle
/signals	SignalsView	Notifications: Love, Replies, Mentions, Follows, Retwists, Secret Twists
/secret-twists	SecretTwistView	End-to-end encrypted messaging inbox, sent, and composer
/about	AboutView	Project README rendered via marked.js
/@:user/social	SocialView	Paginated Followers / Following / Friends with Follow buttons
/@:user/:permlink	TwistView	Single Twist page with full thread context
/@:user	ProfileView	Profile card and chronological Twist list with pagination

4. Feature Overview

4.1 Feed Modes

Twist Stream (default): Fetches only posts with permlinks beginning with tw- that are direct replies to the monthly feed root. This is the native SteemTwist experience — focused, fast, and clearly scoped to the current month.

Understream □: A toggle available on Home, Explore, Profile, and Signals that widens the lens to include all Steem posts and account history. On the Home page, Understream shows all Steem blog posts from followed users; on Explore, it shows the global Steem recent-posts stream.

Firehose □: A real-time WebSocket stream that pushes new Twists and Twist Love events as they are confirmed on-chain. On the Home page the Firehose is filtered to followed Twisters only.

4.2 Twists

- Up to 280 characters with full Markdown support
- Real-time Write / Preview tab switching in the composer
- Edit — re-broadcasts the comment op; the card updates instantly in the feed
- Delete — uses delete_comment if no votes or replies exist; falls back to blanking the body with <deleted> if the post cannot be deleted
- Sort modes: New, Hot, Top
- Pin — one Twist can be pinned to the top of a profile via a custom_json on-chain operation

4.3 Social Graph

- Follow / Unfollow — native Steem follow plugin (custom_json, what:["blog"] / what:[])
- Rich profile cards — avatar, reputation (1–100 scale), bio, location, website (https-only), join date, post count, follower and following counts

- Social pages — paginated Followers, Following, and Friends (mutual follows) lists
- Retwist — resteem any Twist
- Twist Love — upvote at weight 10 000 (100%)

4.4 Signals (Notifications)

The Signals view scans the latest 500 account-history entries and classifies each into one of six signal types:

Signal type	Trigger
Twist Love □	Incoming upvote on a user's Twist
Reply □	New Thread Reply on a user's Twist
Mention □	A Twist body contains @username
Follow □	Another Twister followed this user
Retwist □	Another Twister resteemed a Twist
Secret Twist □	An incoming encrypted private message

An Unread badge counts signals not yet seen. The badge clears when the user visits the Signals page.

5. Live Twists ⚡

5.1 Concept

A Live Twist is a Steem comment whose json_metadata contains a type:"live_twist" field alongside a JavaScript code payload. When a viewer opens the Twist, they see a Run button. Clicking it executes the author's JavaScript inside a tightly sandboxed iframe and renders the output inline in the feed. Live Twists transform static posts into interactive widgets — polls, calculators, charts, blockchain query dashboards, greeting cards, and games — all stored permanently on-chain.

On clients that do not support Live Twists (e.g. Steemit), the Steem comment body falls back to a human-readable message and a link back to SteemTwist.

5.2 On-Chain Format

The json_metadata of a Live Twist comment contains:

```
{ "type": "live_twist", "version": 1, "title": "Click Counter",
  "code": "let n=0; function draw(){ app.render('<button id=b>Clicks:
'+n+'</button>');
    document.getElementById('b').onclick=()=>{n++;draw();}; } draw();" }
```

The code field is capped at 10 KB before publishing. The comment body defaults to "⚡ Live Twist — view on SteemTwist" for cross-client compatibility.

5.3 Sandbox API

Author code communicates with the host page through a message-passing API exposed as the app object inside the sandbox:

Method	Description
app.render(html)	DOMPurify-sanitise and inject HTML into the iframe body; auto-resizes the iframe
app.text(str)	Set body as plain text (capped at 2 000 characters)
app.resize(px)	Manually set iframe height (40–600 px)
app.log(...args)	Append a line to the built-in console panel
app.query(type, params)	Call a read-only Steem API method; result delivered via app.onResult()
app.ask(type, params)	Like app.query() but returns a Promise — safe for concurrent calls
app.action(type, params)	Request a Keychain-signed blockchain operation; shows a confirmation modal
app.onResult(callback)	Register callback(success, data) for query and action results

app.ask() correlates each call by a unique request ID so parallel queries never collide — essential for templates that fire multiple concurrent RPC calls with Promise.all().

5.4 Supported Blockchain Queries

Live Twists can call over 70 read-only Steem API methods via app.query() / app.ask(), spanning discussions, content, accounts, follow graph, chain globals, market data, witness schedule, and block data. All string parameters are capped to 256 characters; all limit parameters are clamped to 1–100; arrays are sliced to 10 entries.

5.5 Supported Blockchain Actions

Live Twists can trigger the following Keychain-signed operations. Each action routes through the parent page for validation and presents an in-page confirmation modal before reaching Keychain:

Action	Key parameters
vote	author, permlink, weight (−10 000 to +10 000)
reply	parentAuthor, parentPermlink, message (≤2 000 chars)
retwist	author, permlink
follow	following (validated Steem username)
unfollow	following (validated Steem username)
transfer	to, amount (decimal), memo (≤2 048 chars), currency (STEEM or SBD)
delegate	delegatee, amount (decimal), unit (SP or VEST)

voteWitness	witness, vote (true/false)
powerUp	to, amount (decimal)
powerDown	amount (decimal)

5.6 Templates Gallery

The Live Twist composer ships with 40 built-in templates in four tabs:

- Simple (10) — Poll, Quiz, Clicker, Calculator, Chart, Expandable, Story, Demo, Explorer, Prototype
- Greetings (10) — Birthday, New Year, Congratulations, Wedding, Graduation, Eid, Christmas, Thank You, Get Well, Anniversary
- Queries (10) — Account Info, Trending Tags, STEEM Price, Hot Posts, Follower Count, Top Witnesses, Chain Stats, Post Viewer, Order Book, Reward Pool
- Actions (10) — Vote on a Post, Reply, Follow/Unfollow, Transfer STEEM/SBD, Delegate SP, Power Up, Vote for Witness, Retwist, Query then Vote, Query then Follow

Each template populates the Title, Body, and Code fields. The Preview tab runs the current code in a live sandbox with auto-resizing output.

5.7 Flag System

Users can flag a Live Twist authored by someone else as harmful. Flagging is implemented as two sequential Keychain operations:

- A −10 000 weight downvote via requestVote
- A reply comment via requestBroadcast whose json_metadata records the reason

The two-step design is required because Steem Keychain rejects vote operations bundled with other operation types in a single requestBroadcast call.

Available flag reasons: Session Hijacking (Cookie Theft), Web Skimming / Formjacking, Storage Theft, DOM-type XSS (Cross-Site Scripting), Phishing Form Insertion, UI Redressing, Cryptojacking, Browser Fingerprinting, Misuse of Location and Camera Information, Tampering with Client-Side Logic, CSRF (Cross-Site Request Forgery), Other.

6. Secret Twists

Secret Twists provide end-to-end encrypted private messaging between Steem accounts using the memo key pair. The encryption and decryption operations are delegated entirely to Steem Keychain, so plaintext never touches SteemTwist's code.

6.1 Send Flow

- Sender composes a message in the Secret Twist composer (unlimited length, Markdown, Write/Preview)
- SteemTwist calls Keychain's requestEncodeMessage with the recipient's memo public key

- The encrypted ciphertext is broadcast as a Steem comment reply to the monthly secret-YYYY-MM root with json_metadata.type = "secret_twist"
- The message is invisible in the regular Twist feed — it only appears in the Secret Twists inbox

6.2 Receive & Decrypt Flow

- Recipient sees a □ signal in the Signals feed
- Opening the Secret Twists view shows the encrypted card
- Clicking Decrypt calls Keychain's requestVerifyKey — the decrypted plaintext is displayed client-side only
- Nested encrypted replies are supported; only the original recipient (non-author) can reply to a Secret Twist

Secret Twists are posted with max_accepted_payout: "0.000 SBD" and allow_votes: false.

7. Economics

SteemTwist deliberately separates social interaction from financial reward. All Twists, Thread Replies, and flag replies are broadcast with:

- max_accepted_payout: "0.000 SBD" — no author reward
- allow_curation_rewards: false — no curation reward
- allow_votes: true — Twist Love still counts as social appreciation

This design prevents the feed from being gamed by vote farmers and ensures that Twist Love reflects genuine engagement rather than economic incentive. Twisters who wish to earn Steem rewards can post through other Steem clients; SteemTwist is explicitly a social layer, not a monetisation layer.

8. Security

8.1 Supply Chain

All CDN-loaded libraries (steem-js, Vue, Vue Router, marked, DOMPurify) carry Subresource Integrity (SRI) hashes and crossorigin="anonymous". A tampered CDN file is rejected by the browser before execution.

8.2 Content Sanitisation

All user-supplied Markdown rendered via v-html passes through DOMPurify.sanitize() before insertion. Profile cover image and profile image URLs are validated to http:// or https:// only inside fetchAccount; cover images are additionally passed through encodeURI() before interpolation into CSS. User-supplied website URLs are validated to https:// only.

8.3 Live Twist Sandbox

The Live Twist sandbox applies ten layered security controls:

- iframe sandbox="allow-scripts" only — null origin, no same-origin access, no form submission, no top navigation
- fetch, XMLHttpRequest, WebSocket, and window.open are overridden to throw inside the sandbox
- DOMPurify is loaded inside the sandbox from CDN and applies LIVE_TWIST_PURIFY_CONFIG — a single shared constant embedded identically in both the viewer and composer preview iframes
- 10 KB code size limit enforced before publish and before Run
- User must click Run — code is never auto-executed on page load
- Keychain is unreachable from inside the sandbox; all app.action() calls route through the parent page validation pipeline
- app.action() parameters are validated (username regex, decimal-only amounts, currency/unit allowlists) and displayed in a styled confirmation modal with HTML-escaped values before any Keychain call
- Parent validates e.origin === "null" and e.source === iframe.contentWindow on every inbound postMessage
- iframeSource is captured synchronously at event-receive time so async RPC callbacks post to the correct window even after Vue re-renders the iframe element
- All app.query() / app.ask() parameters are sanitised before reaching steem.api: strings capped to 256 chars, limits clamped to 1–100, arrays sliced to 10 entries

8.4 localStorage

The pending-pin cache validates author against /^[a-z0-9\-.]{3,16}$/ and permlink against /^[a-z0-9-]{1,255}$/ before use. Tampered or injected cache values are silently discarded.

8.5 ReDoS Prevention

The stripSignalBody function, which applies regex patterns to post bodies fetched from the blockchain, caps its input to 10 000 characters before processing to prevent Regular Expression Denial of Service from crafted inputs.

9. Component Reference

SteemTwist is composed of 15 Vue 3 components. Key architectural notes:

- LIVE_TWIST_PURIFY_CONFIG is defined once and embedded into both the viewer sandboxDoc and the composer buildSandboxDoc, guaranteeing identical sanitisation in both contexts
- handleQueryRequest and handleActionRequest are defined once in LIVE_TWIST_HANDLER_MIXIN and spread into both LiveTwistComponent and LiveTwistComposerComponent, ensuring identical query/action behaviour in the live viewer and the composer preview

Component	Description
AppNotificationComponent	Toast notifications — auto-dismiss after 3.5 s for non-errors
AuthComponent	Keychain login / logout modal

UserProfileComponent	Profile card: avatar, reputation badge, bio, meta row, stats grid
LoadingSpinnerComponent	Animated loading spinner
ReplyCardComponent	Thread Reply with Love / Retwist / Reply / Edit / Delete actions
ThreadComponent	Lazy-loads and enriches nested replies; auto-expands two levels
LiveTwistComponent	Live Twist viewer: Run/Stop, sandboxed iframe, query/action bridge
TwistCardComponent	Full Twist card: action bar, body, Edit, Delete, inline flag panel
LiveTwistComposerComponent	Live Twist editor: Code/Preview tabs, Templates gallery, Publish
TwistComposerComponent	Twist / Live Twist tab composer with Write/Preview
SignalItemComponent	Signal row: type icon, preview text, timestamp, View link
UserRowComponent	Twister row with optional Follow/Unfollow button
SecretTwistComposerComponent	Secret Twist composer: recipient, textarea, Write/Preview, Send
SecretTwistCardComponent	Secret Twist card: Decrypt, Reply (non-author only), nested replies

10. Global State

The root App component provides the following reactive values to all child components via Vue's provide/inject mechanism:

Key	Type	Notes
username	ref<string>	Logged-in username; persisted in localStorage
hasKeychain	ref<boolean>	Detected on mount via polling; gating all signing actions
notify	function	notify(msg, type) — type: error | success | info
unreadSignals	ref<number>	Badge count; recomputed on navigation to Signals
refreshUnreadSignals	function	Called on login and navigation
understreamOn	ref<boolean>	Understream toggle; persisted in localStorage
toggleUnderstream	function	Flips and persists understreamOn

11. Deployment & Operations

11.1 Hosting

Push the four files to any static host and enable serving. Because Vue Router uses hash history, no server-side URL rewriting is required. GitHub Pages on the main branch root is the canonical deployment target.

11.2 Monthly Setup

At the start of each calendar month the @steemtwist account must publish two root posts:

Permlink	Body
feed-YYYY-MM	SteemTwist feed — Month YYYY
secret-YYYY-MM	SteemTwist secret feed — Month YYYY

Both posts use parent_permlink: steemtwist, empty title, and empty parent_author.

11.3 Forking

The TWIST_CONFIG object in blockchain.js centralises all deployment-specific constants: ROOT_ACCOUNT, ROOT_PREFIX, SECRET_ROOT_PREFIX, TAG, POST_PREFIX, TAGS, and DAPP_URL. A fork need only update these values and publish its own monthly root posts to operate a fully independent feed.

12. Summary

SteemTwist demonstrates that a full-featured social platform — with personalised feeds, real-time streaming, encrypted messaging, programmable interactive posts, and a rich notification system — can be delivered as four static files with no proprietary backend, relying entirely on a public blockchain for storage and a browser extension for key management.

The architecture is intentionally minimal: every dependency is a well-audited open-source library loaded with SRI verification, every signing operation is delegated to the user's own Keychain, and every piece of content is owned by the author's Steem private key — not by SteemTwist.

The project is open source under the MIT License. Source code and deployment instructions are available at **https://github.com/puncakbukit/steemtwist**. The live application is accessible at **https://puncakbukit.github.io/steemtwist**.